

Big-O

What's the big picture?

Big-O is a technique that lets you **quickly understand** an algorithm's **efficiency**.

"This function has a big-O of N^2 which means that if you pass in N items to it, it will take roughly N^2 steps to process its data."

Big-O is **not exact** - instead, it gives rough estimates in terms of well-known functions like $\log_2(N)$, N , $N \cdot \log_2(N)$, or N^2 .

We use the Big-O approach to quickly compare different algorithms and pick the best one.



Uses:

Picking the best algorithm (so you don't look bad at work), job interviews, etc.

How Fast is That Algorithm?

Right! We could measure the time it takes for an algorithm to run.



But that has flaws!
What are they?

Carey: "My algorithm finished in 31 seconds."

Cedric: "Mine finished in 30 seconds, it's better!"

Carey: "Not so fast! How fast is your PC? Mine is 1GHZ."

Cedric: "Err... Mine is 3GHZ."

Carey: "Aha - so my algorithm is really almost 3x faster!"

Cedric: "Sigh. Carey's right again."

How Fast is That Algorithm?

Ok, so simply measuring the run-time of an algorithm isn't really all that useful.

What if instead we measure an algorithm based on:
how many computer instructions it takes
to solve a problem of a given size

Carey: "My algorithm took 370 million instructions to sort 1,000 numbers."

Cedric: "Dude - you SUCK! Mine only took only 5 million instructions, it's better!"

Carey: "Not so fast grasshopper! Mine might be slower on 1,000 numbers, but what if we sort 1 million numbers?"

Cedric: "Hmm. I don't know - I haven't tried."

How Fast is That Algorithm?

Hmmm. What else could we do?

Right! What if we specify
the number of instructions used by an algorithm
as a function of the size of the input data.

"I'm trying to sort N numbers."

"Algorithm A takes $5 \cdot N^2$ instructions to do that."

"Algorithm B takes $37,000 \cdot N$ instructions to do that."

Now we can predict which algorithm will be faster
for any value of N !

How Fast is That Algorithm?

"I'm trying to sort **N numbers**."

"**Algorithm A** takes $5 \cdot N^2$ instructions to do that."

"**Algorithm B** takes $37,000 \cdot N$ instructions to do that."

Ok, what if we're sorting **1,000** numbers:

"**Algorithm A** takes **5M** instructions."



"**Algorithm B** takes **37M** instructions."

Ok, what if we're sorting **10,000** numbers:

"**Algorithm A** takes **500M** instructions."

"**Algorithm B** takes **370M** instructions."



Ok, what if we're sorting **1 million** numbers:

"**Algorithm A** takes **5 trillion** instructions."

"**Algorithm B** takes **37 billion** instructions."



How Fast is That Algorithm?

"I'm trying to sort **N numbers**."

"**Algorithm A** takes **$5 \cdot N^2$** instructions to do that."

"**Algorithm B** takes **$37,000 \cdot N$** instructions to do that."

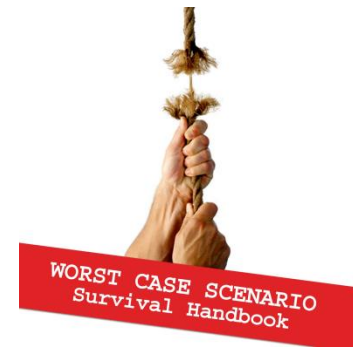
Cool! When we measure this way, we get two benefits:

1. We can **compare** two algorithms for a **given sized input**.
2. We can **predict** the performance of those algorithms when they are **applied to less or more data**.

This is the idea behind the "**Big-O**" **concept** used in Computer Science.

7 Big-O: The Concept

The Big-O approach measures an algorithm by the **gross number of steps** that it requires to process an input of **size N** in the **WORST CASE scenario**.



We could be specific and say:
"Algorithm X requires $5N^2+3N+20$ steps to process N items."

But with Big-O, we **ignore** the **coefficients** and **lower-order terms** of the expression...

So we'd say:
"The Big-O of Algorithm X is N^2 ."

While less specific, this still gives us an overall impression of an algorithm's **worst-case** efficiency.

Big-O: The

Big-O Idea: Use simple functions like n^2 , n^3 , etc. to convey how many operations an algorithm will perform to process n items in the worst case.

This is pronounced:

"oh of n squared"

"That sorting algorithm is $O(n^2)$, so to sort $n=1000$ items it requires roughly 1 million operations."

"That sorting algorithm is $O(n \cdot \log_2 n)$, so to sort $n=1000$ items requires roughly 10,000 operations."

This allows us to easily compare two different algorithms:

"Algorithm A is $O(n^2)$, which is much slower than algorithm B which is $O(n \cdot \log_2 n)$."

Big-O

So how do we compute the Big-O of a function?

First, we need to determine the number of **operations** an algorithm performs. Let's call this **$f(n)$** .

By **operations**, we mean any of the following:

1. Accessing an item (e.g. an item in an array)
2. Evaluating a mathematical expression
3. Traversing a single link in a linked list, etc...

Let's see how to evaluate the number of operations for a simple example...

Big-O - How to Compute $f(n)$

```
int arr[n][n];
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        arr[i][j] = 0;
```

Compute $f(n)$, the # of critical operations, that this algorithm performs?

$$f(n) = 1 + n + n + n + n^2 + n^2 + n^2$$

$$f(n) = 3n^2 + 3n + 1$$

1. Our algorithm initializes the value of i once.
2. Our algorithm performs n comparisons between i and n .
3. Our algorithm increments the variable i , n times.
4. Our algorithm initializes the value of j , n different times.
5. Our algorithm performs n^2 comparisons between j and n .
6. Our algorithm increments the variable j , n^2 times.
7. Our algorithm sets $arr[i][j]$'s value n^2 times.

Now that we have $f(n)$, we can compute our algorithm's Big-O.

Big-O - The Complete Approach

Here are the steps to compute the Big-O of an algorithm:

1. Determine how many steps $f(n)$ an algorithm requires to solve a problem, in terms of the number of items n .
2. Keep the most significant term of that function and throw away the rest. For example:
 - a. $f(n) = 3n^2 + 3n + 1$ becomes $f(n) = 3n^2$
 - b. $f(n) = 2n \log(n) + 3n$ becomes $f(n) = 2n \log(n)$
3. Now remove any constant multiplier from the function:
 - a. $f(n) = 3n^2$ becomes $f(n) = n^2$
 - b. $f(n) = 2n \log(n)$ becomes $f(n) = n \log(n)$
4. This gives you your "big oh":
 - a. $f(n) = 3n^2 + 3n + 1$ is therefore $O(n^2)$
 - b. $f(n) = 2n \log(n) + 3n$ is therefore $O(n \log(n))$

Big-O Simplification

Actually, if you think about it, there's **no need** to compute the **exact** $f(n)$ of an algorithm...

Why? Because we end up **throwing away** all of the lower-order terms and coefficients anyway!

All you need to do is focus on the most **frequently occurring operation(s)** to save time!

```
int arr[n][n];
```

```
for ( int i = 0; i < n; i++ )  
  for ( int j = 0; j < n; j++ )
```

```
    arr[i][j] = 0;
```

$f(n) = n^2$

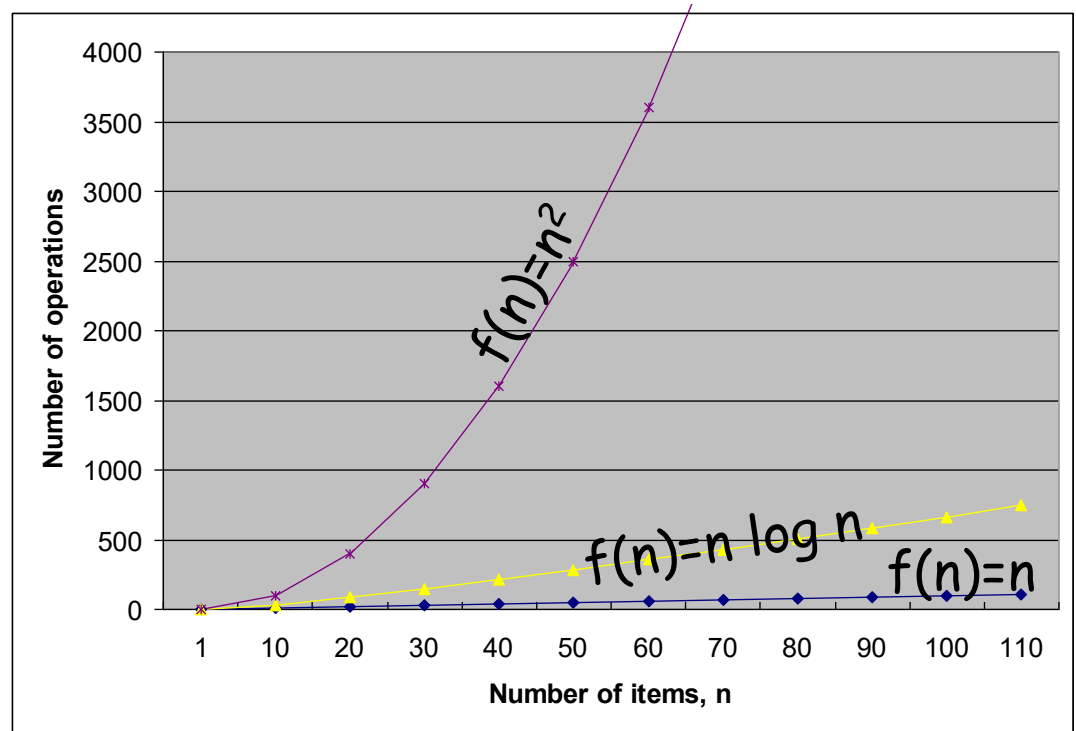
Our algorithm is $O(n^2)$.

Big-O

So if I say: "This algorithm is $O(n^2)$."

I really mean: "To process n items, this algorithm requires roughly n^2 operations."

By using only the most significant term (e.g. n^2 from $2n^2+3n+1$) We can quickly obtain a rough approximation of how many steps our algorithm will take to process n items.



Big-O Complexity

$\log_2 n$	N	$n \log_2 n$	n^2	n^3	2^n
3	10	30	100	1000	1000
6	100	600	10,000	1,000,000	10^{30}
9	1,000	9,000	1,000,000	1,000,000,000	10^{301}
13	10,000	130,000	100,000,000	10^{12}	WOW!
16	100,000	1,600,000	10^{10}	10^{15}	WOW!
19	1,000,000	19,000,000	10^{12}	10^{18}	WOW!

What if you wanted to use an $O(n^3)$ algorithm to sort a million numbers? Your algorithm would require roughly 1,000,000,000,000,000,000 steps!

But an $O(n \log_2(n))$ algorithm would use only 19,000,000 steps!

Big-O Complexity

1,000,000,000,000,000,000 vs 19,000,000!

GREAT PROGRAMMERS know that the choice of algorithm makes all the difference in the world.

NOT-SO-GREAT programmers think that you can tweak a poor algorithm to make it better!

Say you improve an $O(n^3)$ algorithm from $f(n) = 5n^3$ steps to $f(n) = 1.5n^3$ steps. For $n=1,000,000$, that reduces the number of steps from 5,000,000,000,000,000,000 to 1,500,000,000,000,000,000.

(Big deal... so it'll take 1.5 years to run instead of 5 years)

However, if you can find an algorithm that's $O(n \cdot \log n)$ steps, say $f(n) = 10 \cdot n \log n$, you can solve the problem in 190,000,000 steps.

(Which will take just a few seconds or less on a modern PC!)

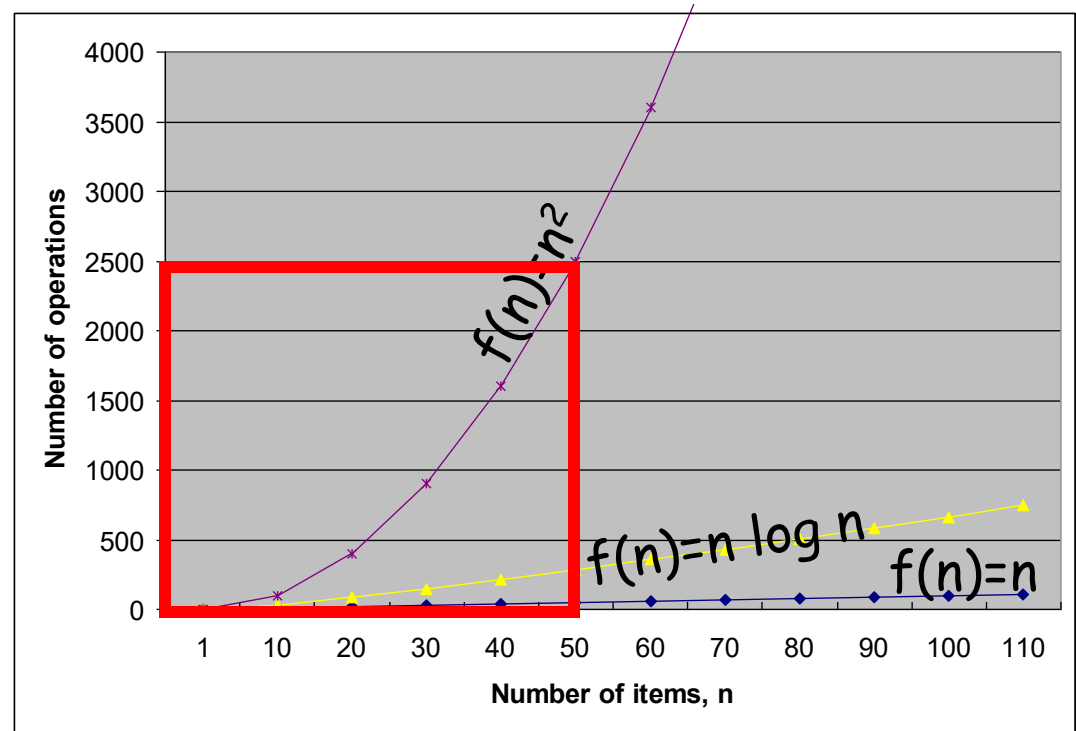
Don't be a Big-O NUT!

When you're writing a program that operates on a **large number of items**, evaluating Big-O is key. It can mean the difference between a usable program and an unusable one.

But what if you have a small number of items, e.g. $n < 50$?

In this case, all of your algorithms require only a small number of steps.

In such situations (when you know n is small), forget which Big-O is better and choose the easiest-to-program algorithm. **It'll save you lots of headaches.**



Find the Big-O Challenge, Part 1

```
for (
    sum
```

Our outer loop runs exactly n times...

```
for (int i = 0; i < q; i++)
    for (int j = 0; j <
        sum++;
```

the outer loop runs q times...

```
for (int
    for (i
        sum
```

Each time, the inner loop runs $\log_3(n)$ iterations.

Our first loop runs exactly n^2 times.

```
k = n;
while (k > 1)
{
```

```
    sum
    k =
```

Our second loop runs exactly n^3 times...

```
}
```

```
    for (i=0; i < n*n*n; i++)
        sum += i;
```

So overall the code runs $n^2 + n^3$ iterations. So it's $O(n^3)$.

all, the O is $\log_3(n)$.

Find the Big-O Challenge, Part 1a

```
int searchArray(int arr[], int n, int forValue)
{
    for ( int i = 0; i < n; i++ )
    {
        if (arr[i] == forValue)
            return i;
    }

    return -1; // not found
}
```

Our loop runs at most n times, so this function is $O(n)$.

```
void addItemToEndOfArray(int arr[], int &n, int addMe)
{
    arr[n] = addMe;
    n = n + 1;
}
```

We run the same, fixed number of steps no matter what n 's value is. So it's $O(1)$.

Big-O... my

Sometimes you'll run into an algorithm that isn't so clear-cut. For example, what's the Big-O of `mystery`?

It's clear that the outer loop runs n times, but what about the inner loop?

When $q = 0$, the inner loop runs 0 iterations.

When $q = 1$, the inner loop runs 1 iteration.

When $q = 2$, the inner loop runs 2 iterations.

...

When $q = n-1$, the inner loop runs $n-1$ iterations.

```
void mystery(int n)
{
    for (int q = 0; q < n; q++)
    {
        for (int x = 0; x < q; x++)
        {
            cout << "Waahoo!";
        }
    }
}
```

So what's the Big-O?

$$f(n) = \frac{n^2 - n}{2}$$

$$O(n^2)$$

So the `cout` statement will run a total of:

0 times + 1 time + 2 times + 3 times + ... + $n-1$ times

And if you recall a clever trick, this is equal to: $\frac{n \cdot (n-1)}{2}$

Big-O: Such Ugly Math! 😊

But we're not Math geeks... We're CS geeks! So here's a way to address these situations without formulas!

Locate all loops that **depend** on **n** and determine the **maximum** number of iterations.

Turn these loops into **fixed** loops.

But this loop runs a variable number of iterations and depends on **n**.

This outer loop always runs exactly **n** iterations (a fixed #).

There's nothing to change.

Let's just round that up to **n**, and then make our loop a fixed loop!

```
func1(int n)
{
  for ( int i = 0; i < n; i++ )
    for (int j=0; j < n; j++)
      cout << j;
}
```

$O(n^2)$

In that case, this loop will run up to **n²** iterations - Let's update it!

```
for ( int x = 0; x < n; x++ )
  for (int j=0; j < n*n; j++)
    cout << "p!";
```

$O(n^3)$

Find the Big-O

```
for (
  for (
```

```
sum++;
```

```
for (int
  for (i
    sum
```

```
for (int
  for (
    for (
      sum++;
```

Each time the outer
runs once...

As before, variable i possible value
value is going to be n . basically n .

This line runs the Circ
constructor n times!

As before, it's largest
value that is q^2 .

our inner runs q^2 times. Let's
maximum possible value. $O(n^2)$. it's

So let's replace
this i with n .

So... Let's replace
these i 's with p 's.

That makes the
total count
 $n * n^2 * n^2$
or $O(n^5)$

$J < p * p$
 $0; k < p$

$O(p^4)$

But we're not done
yet! This one too!

$O(n * \log_2(n))$

Multi-in

The number of people, p , and the number of foods, f , are completely independent.

The number of CS students, c , and EE students, e , are also independent.

```
void buffet(string people[], int p,
            string foods[], int f)
{
    int i, j;

    for (i=0; i < p ;i++)
        for (j=0; j < f ;j++)
            cout << people[i] << " ate "
                << foods[j] << endl;
}
```

 $O(p * f)$

```
void tinder(string csmajors[], int c,
            string eemajors[], int e)
{
    for (int i=0; i < c ;i++)
        for (int j=0; j < c ;j++)
            cout << csmajors[i] << " dates "
                << csmajors[j] << endl;

    for (int k=0; k < e ;k++)
        cout << eemajors[k] << " sits ";
}
```

 $O(c^2 + e)$

In th



What? $O(c^2 + e)$ and not just $O(c^2)$?
 Correct! Even though c^2 seems bigger than e , we must include both independent variables in our Big-O!

Big-O, we sizes.

Big-O for Multi-input Algorithms

Why must we include both variables in the Big-O
(even if one is higher-order than the other)?

Because either variable could **dominate** the other!

```
void tinder(string csmajors[], int c,
            string eemajors[], int e)
```

```
{
  for (int i=0; i < c ;i++)
    for (int j=0; j < c ;j++)
      cout << csmajors[i] << " dates "
        << csmajors[j] << endl;
```

```
  for (int m=0; m < c ; m++)
    cout << csmajors[m] << " is still a nerd";
```

```
  for (int k=0; k < e ;k++)
    cout << eemajors[k] << " sins at home"
```

```
}
```



ALERT! But... don't

forget that you still must

eliminate lower-order

terms for each

independent variable!

c^2 iterations.

c were 100...

~~c iterations.~~

are 10 billion?

e iterations.

$O(c^2 + e)$

Find the Big-O Challenge, Part 3

```
void bar( int n, int q )
{
    for (int i=0 ; i < n*n ; i++)
    {
        for (int j=0 ; j < q ; j++)
        {
            cout << "CS!";
        }
    }
}
```

We perform n outer iterations.

```
void burp( int n )
{
    for (int i=0 ; i < n ; i++)
    {
        cout << "Burp!";
    }
}
```

So n iterations plus a one-time run of q iterations:
 $O(n + q)$

```
void bletch( int n, int q )
{
    for (int i=0 ; i < n ; i++)
        cout << "Muahahaha!";
}
```

And the one time when i is equal to $n/2$...

```
void barf( int n, int q )
{
    for (int i=0 ; i < n ; i++)
    {
        if (i == n/2)
        {
            for ( int k = 0; k < q ; k++ )
                cout << "Muahahaha!";
        }
        else
            cout << "Burp!";
    }
}
```

...we run this inner loop which has q iterations.

The STL

Remember the STL - **stacks**, **queues**

Well, these classes use **algorithms**

And these **algorithms**

Hmmm. Does that remind you of a search algorithm that we learned on the first day of class?



```
void inDict(set<string> d)
{
    if ( d.find(w) == d.end() )
        cout << w << " isn't in dict\n";
}
```

```
void otherFunc(vector<int> &vec)
{
    vec.push_back(123);
    vec.erase( vec.begin() );
}
```

Any guesses why it takes n steps to delete the first item from a **vector** containing n items?

Well, if we want to find a word in a **set** that contains n words (a dictionary), it takes $O(\log_2(n))$ steps!

Want to add a value to the end of a **vector** holding n items, it takes just **one step**, so it's $O(1)$!

And if we want to delete the 1st value from a **vector** containing n items, it takes a whopping n steps, making it $O(n)$!

The STL and Big-O

It's important to understand the complexity of STL operations (e.g. `push_back`, `erase`) for

... because without knowing this

we can't compute the Big-O of code that uses the STL classes!

Well, to search a **set** of **n items** for a single value requires $\log_2(n)$ steps...

For example...

If we write a loop of our own that runs **D times**...

And each iteration of our loop searches for an item in a **set** holding **n items**...

Then the Big-O of the whole loop would be $O(D * \log_2(n))$.

```
void inDict(set<string> & d, string w)
{
    if ( d.find(w) == d.end() )
        cout << w << " isn't in dictionary!";
}
```

And we repeat this search operation **D** different times...

```
for (int i=0; i < D; i++)
    inDict( dict, doc[i] );
}
```

Com

Ok, our vector contains q values...

L

of Algorithms that use STL

And as such our loop runs

And each time our loop

```
void printItems( vector<int> & v )
{
    int q = v.size();
    for ( int i = 0; i < q ; i++ )
    {
        int a = v[0]; // get 1st item
        cout << a; // print it out
        v.erase( v.begin() ); // erase 1st item
        v.push_back(a); // add it to end
    }
}
```

2. Erase the **first** item in the vector: the Big-O of

3. Add the item to the **end** of the vector: the Big-O of adding an item to the end of a vector is $O(1)$.

Hint:

Total steps performed during our loop:

$$q * (1 + q + 1)$$

So our total Big-O is:

$$O(q^2)$$

The STL vector

Insert at the top/middle:	$O(n)$
Insert at the end:	$O(1)$
Delete an item from top/middle:	$O(n)$
Delete an item from the end:	$O(1)$
Access an item:	$O(1)$
Finding an item:	$O(n)$

STL and Big Oh Cheat Sheet

When describing the Big-O of each **operation** (e.g. **insert**) on a **container** (e.g., a **vector**) below, we assume that the container holds **n items** when the operation is performed.

Name: **list**

Purpose: Linked list

Usage: `list<int> x; x.push_back(5);`

Inserting an item (top, middle*, or bottom): **$O(1)$**

Deleting an item (top, middle*, or bottom): **$O(1)$**

Accessing an item (top or bottom): **$O(1)$**

Accessing an item (middle): **$O(n)$**

Finding an item: **$O(n)$**

*But to get to the middle, you may have to first iterate through **X** items, at cost **$O(X)$**
 If instead of holding **n** items, a container holds **p** items, then just replace "**n**" with "**p**" when you do your analysis.

So if our loop runs a total of q iterations...

and each iteration we insert at a cost of $\log_2(q)$ into our set...

Then our total cost is $q * \log_2(q)$

```
int main( )
{
    set< int > nums;
    for (int i=0; i < q ; i++)
        nums.insert( i );
}
```

And that's the correct answer!

Algorithm

Ok, I'll assume that our set always has q items in it.

That means that each time we insert an item into our set it takes $\log_2(q)$ steps.

So... should we use as the cost of inserting an item into the set?

number into a set with n items takes $\log_2(n)$ steps.

And in fact, our set will have a different number of items during each iteration of the loop!

Deleting an item:
 $O(\log_2 n)$

$O(\log_2 n)$
 $O(\log_2 n)$

Well, assuming our vector has p items and we delete one item per iteration, our loop runs for p steps.

So there are p total loop iterations, and during each iteration, we perform p steps to delete the first item:
 $O(p^2)$

```
// Assume p items in vector v
void clearFromFront(vector<int> &v)
{
    while ( v.size() > 0 )
    {
        v.erase( v.begin() ); // erase 1st item
    }
}
```

Hints:

The STL vector

Insert at the top/middle:	$O(n)$
Insert at the end:	$O(1)$
Delete an item from top/middle:	$O(n)$
Delete an item from the end:	$O(1)$
Access an item:	$O(1)$
Finding an item:	$O(n)$

During each step, we delete the vector's first item.

Let's assume the vector always has p items, no matter what.

Cost of deleting first item: $O(p)$

Challenge, Part 4

So we perform q^2 total loop iterations, and during each iteration, we perform $\log_2(q^2)$ steps to insert an item:
 $O(q^2 \log_2(q^2))$

```
// Assume s starts out empty
void addItems(set<int> &s, int q)
{
    for (int i=0; i < q*q; i++)
        s.insert(i);
}
```

Ok, our loop runs through q^2 total iterations.

And the cost of adding a single item to a set with q^2 items is... $O(\log_2(q^2))$

As before, to compute the cost of an STL operation, we assume the set always holds its max # of items.

In this case, the set will eventually hold q^2 items - that's our max!

Delete an item from the end:	$O(1)$
Access an item:	$O(1)$
Finding an item:	$O(n)$

The STL set

Inserting a new item:	$O(\log_2 n)$
Finding an item:	$O(\log_2 n)$
Deleting an item:	$O(\log_2 n)$

Well, assuming our vector has z items and we delete one item per iteration, our loop runs for z steps.

So there are z total loop iterations, and during each iteration, we perform 1 step to delete the last item:
 $O(z)$

```
// Assume  $z$  items in vector v
void clearFromBack(vector<int> &v)
{
    while ( v.size() > 0 )
    {
        v.pop_back(); // erase last item
    }
}
```

Hints.

The STL vector

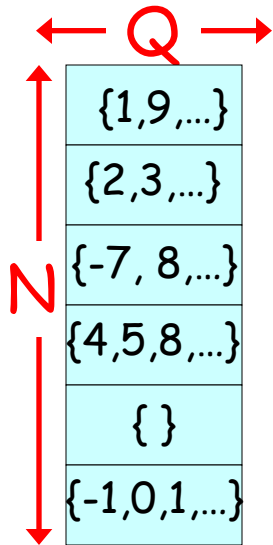
Insert at the top/middle:	$O(n)$
Insert at the end:	$O(1)$
Delete an item from top/middle:	$O(n)$
Delete an item from the end:	$O(1)$
Access an item:	$O(1)$
Finding an item:	$O(n)$

During each iteration, we delete the vector's last item.

Let's assume the vector always has z items, no matter what.

Cost of deleting last item: $O(1)$

Find the Big-O Challenge, Part 5



I have a vector of sets of ints:

```
vector< set<int> > v;
```

You may assume vector **v** has **N** total sets in it.
You may assume that each set has an avg of **Q** items.

Questions:

What is the Big-O of determining if the first set, $v[0]$, contains the value 7?

What is the Big-O of determining if any set in v has a value of 7?

What is the Big-O of determining the number of even values in all of v ?

What is the Big-O of finding the first set with a value of 7 and then counting the number of even values in that set?

The STL set

Inserting a new item:	$O(\log_2 n)$
Finding an item:	$O(\log_2 n)$
Deleting an item:	$O(\log_2 n)$